

ESCUELA TÉCNICA
SUPERIOR DE INGENIEROS
INFORMÁTICOS

UNIVERSIDAD POLITÉCNICA DE MADRID

Fundamentos de Videojuegos

ITERACIÓN 2

Jose Luis Prado Sierra - 220070
Juan Francisco Aranda Rodríguez - 240259
Javier Sánchez Valentín - 240167
David Sobrino Rodríguez - 240022
David Sierra Lanaquera - 240248
Fernando Santana Riñón- 240251

1. Propuesta e historia

Puppeteer Stories es un videojuego en el que eres un titiritero que controla una marioneta y a la vez interaccionas con el entorno (un backstage con diferentes elementos del mundo del teatro) para pasar niveles de tipo plataforma y puzle, creando en cada uno una pequeña historia distinta. El **jugador** es un títere aburrido que crea sus pequeñas historias, por lo que el control de la marioneta y el estado del entorno dependen ambos de él. El personaje principal es la **marioneta**, que controla el jugador. Existe la posibilidad de que haya diferentes enemigos como en **Little Nightmares** que la marioneta tenga que evitar o distraer para llegar a su objetivo.

2. Género y referencias

El juego es una mezcla de los géneros plataformas y puzle.
Mezcla inspiraciones de:

- Little Nightmares.
- It Takes Two.
- Little Big Planet.
- Cut The Rope.

3. Jugadores objetivo

El juego, al ser un juego de plataformas con puzles y controles sencillos, está enfocado a un público genral donde se pueda divertir gente de diversas edades y experiencia jugando videojuegos. El tono y temático del juego es apta para todos los públcos y no se tocan temas sensibles.

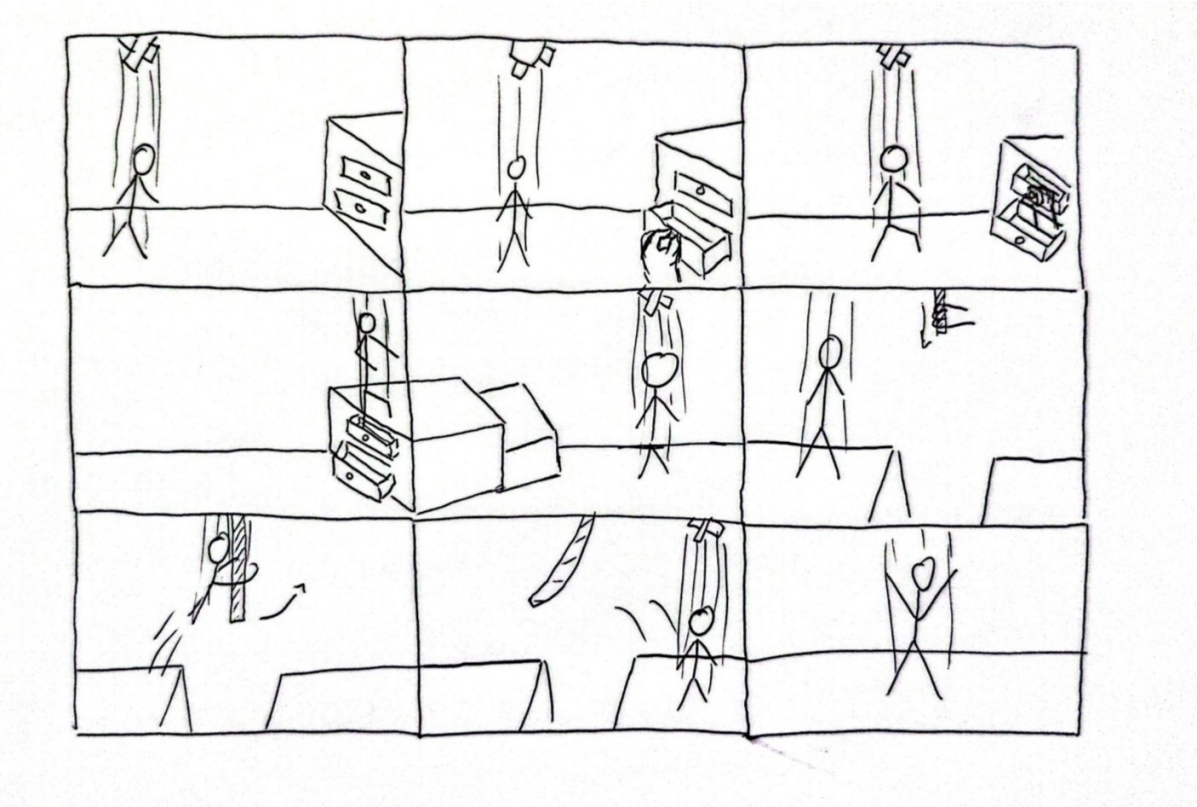
4. Motivadores

El objetivo del juego es pasarse los distintos niveles para descubrir cada historia. Resolver los puzles ayudará a que el jugador se enfrente a nuevos desafíos que lo motiven a seguir jugando. También habrá checkpoints con el objetivo de que al jugador no se le haga repetitivo si muere varias veces.

5. Mecánica principal

Mover al personaje con el teclado y mover elementos del entorno con el ratón para resolver diversos obstáculos y puzles dentro de cada nivel. Se moverán elementos del entorno como cajas o cajones, arrastrándolos por un camino para usarlos de plataforma, permitiendo al jugador acceder a zonas que previamente no podía.

6. Storyboard



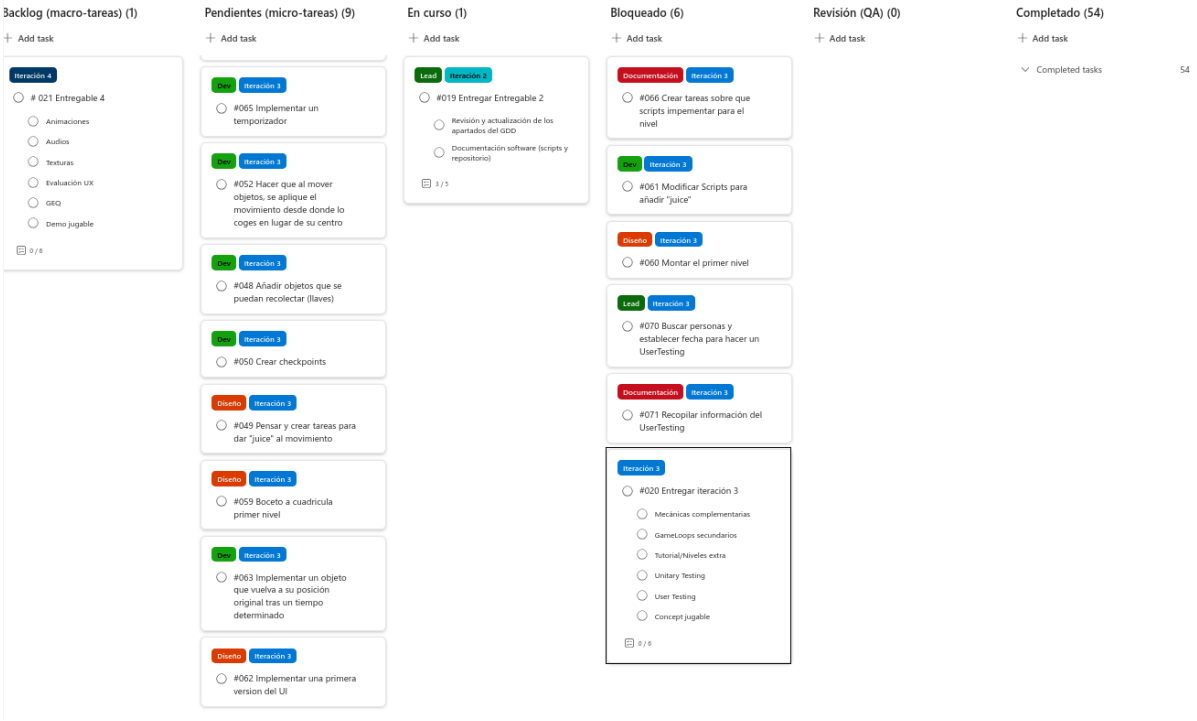
7. Organización

7.1. Roles

- **Team Lead:** Encargado de coordinar la planificación, establece fechas, asigna tareas, supervisa el proyecto y toma la decisión final en caso de duda o conflicto.
- **Designer:** Encargado de diseñar las mecánicas, niveles, y escenarios.
- **Developer:** Encargado de programar y hacer que funcione lo diseñado por los Designer.
- **QA:** Encargado de probar el producto en busca de fallos ya sean de código, experiencia, calidad,etc. Además de hacer UnitTest.
- **Document Controller:** Organiza y mantiene actualizada toda la documentación.
- Jose Luis Prado: Team Lead, Developer y QA
- Juan Francisco Aranda: Developer y Designer

- Javier Sánchez: Developer y Designer
- David Sobrino: QA y Developer
- David Sierra: Document Controller y QA
- Fernando Santana: Developer y Document Controller

7.2. Kanban



8. Quality Assessment

9. Documentación de Scripts y Repositorio

Scripts:

- `Player2_5D.cs`: controla el movimiento del jugador, el salto, su gravedad y la detección del suelo.
- `MovingPlatform.cs`: mueve una plataforma en el eje Z que cambia de dirección al llegar a los límites.
- `DragableObjects2_5D.cs`: permite arrastrar con el ratón objetos con tag `Drag`.

- `DragDrawers.cs`: arrastra objetos con tag `Drawer` mediante un muelle y limita su recorrido en X.
- `FollowHandle.cs`: hace que el cajón siga al handle manteniendo su desfase inicial.
- `FallDownWhenClick.cs`: activa la gravedad de un bloque al hacer clic sobre él.
- `Fusion.cs`: prototipo que une temporalmente un objeto arrastrado con el jugador mediante un `FixedJoint`.
- `NewTestScript.cs`: pruebas automatizadas de movimiento, salto, plataformas y arrastre.

Estructura del Proyecto:

```
Game/
‘-- Assets/
    |-- Code/
    |   ‘-- Movement/        # Player2_5D, MovingPlatform,
    |                        # DragableObjects2_5D, DragDrawers,
    |                        # FollowHandle, FallDownWhenClick,
    |                        # Fusion, Game.asmdef
    |-- Materials/           # Arrastrable.mat, Color Hielo.mat,
    |                        # Resbaladizo.physicMaterial
    |-- Prefabs/             # Player, MovingPlatform, Handle, Drawer,
    |                        # DrawerBox, FinishedDrawer, Draggable,
    |                        # FallWhenClickBlock
    |-- Scenes/
    |   |-- GameLab/         # GameLab2.5D.unity
    |   ‘-- TestLevels/      # TestLevel1.unity
    ‘-- Tests/               # NewTestScript.cs, Tests.asmdef
```

10. Documentación de IA Generativa

- Jose Luis: DeepSeek v4.1 Flash para comprender y mejorar el archivo de movimiento del personaje. Todo lo que ha generado ha sido revisado línea a línea, comentado a mano y ajustado a la necesidad real del juego, eliminando código innecesario o demasiado específico.
- Juan Francisco: DeepSeek Chat para la implementación inicial del movimiento del personaje, contrastando con información de tutoriales de Reddit en r/unity, tomando diferentes trozos de código poco a poco.
- David Sobrino: En el script fusion para arreglar problemas de que los joint. Se hacian lejos del objeto con el cual se tenia que hacer joint.
- Fernando: En los scripts DragDrawers.cs y FollowHandle.cs. También para establecer los limites de movimiento del jugador y para que el jugador conserve el movimiento en el salto si no se hace ningun input.
- David Sierra: Gemini, En los test en busca del problema sobre porqué no se detectaban los input al ejecutarlos.

11. Versiones GDD

- iteration1.pdf: Versión 1 del GDD. Contiene: Propuesta e historia, género y referencias, jugadores objetivo, motivadores, mecánica principal, storyboard, y organización (roles y kanban).
- iteration2.pdf: Versión 2 del GDD. Se mejoran los apartados de jugadores objetivos y mecánica principal. Se añade: Quality Assessment, Documentación de Scripts y Repositorio (Scripts y Estructura del proyecto), Documentación de IA Generativa, y Versiones GDD